

Beta BASIC v2.2 Complete User Manual

1. Core ZX BASIC Commands (Sinclair Standard)

These are the foundational commands of classic 8-bit programming.

Command	Syntax	Description & Example
PRINT	<code>PRINT expr [; or ,]</code>	Prints text/math. Use <code>;</code> to stay on the same line, <code>,</code> to tab. <code>PRINT "SCORE: "; S</code>
LET	<code>[LET] var = expr</code>	Assigns a value. The <code>LET</code> keyword is optional. <code>LET LIVES = 3</code> or <code>LIVES = 3</code>
INPUT	<code>INPUT ["Prompt";] var</code>	Pauses and asks user for input. <code>INPUT "WHAT IS YOUR NAME? "; N\$</code>
CLS	<code>CLS</code>	Clears the text and graphics screen to the current paper color. <code>CLS</code>

BORDER	<code>BORDER color [,width]</code>	Sets the border frame color (0-7).
	<code>BORDER 1</code>	
INK / PAPER	<code>INK col / PAPER col</code>	Sets text foreground (INK) and background (PAPER) colors.
	<code>PAPER 7 : INK 0</code>	
BRIGHT / FLASH	<code>BRIGHT n / FLASH n</code>	Toggles high intensity or blinking text (1 = On, 0 = Off).
	<code>FLASH 1</code>	
GOTO	<code>GOTO line_number</code>	Jumps program execution to a specific line number or label.
	<code>GOTO 100</code>	
GOSUB / RETURN	<code>GOSUB line ... RETURN</code>	Jumps to a subroutine and returns when <code>RETURN</code> is hit.
	<code>GOSUB 5000</code>	

FOR / NEXT	<code>FOR v = x TO y [STEP z]</code>	Creates a counted loop. Must be closed with NEXT.
-------------------	--	---

`FOR I = 1 TO 10 STEP 2 : NEXT I`

IF / THEN / ELSE	<code>IF cond THEN stmt ELSE</code>	Conditional execution.
-------------------------	---	------------------------

`IF HP < 1 THEN PRINT "DEAD" ELSE
PRINT "ALIVE"`

READ / DATA	<code>DATA val1 / READ var</code>	Reads sequential embedded data into variables.
--------------------	---------------------------------------	--

`DATA 10, 20 : READ A`

RESTORE	<code>RESTORE [line_or_label]</code>	Resets the <code>DATA</code> pointer to the start, or a specific line.
----------------	--	--

`RESTORE "LEVEL_2"`

PAUSE	<code>PAUSE frames</code>	Pauses execution (50 frames = 1 second).
--------------	---------------------------	--

`PAUSE 100`

BEEP	<code>BEEP duration, pitch</code>	Plays a simple tone. Pitch is in semitones relative to middle C.
	<code>BEEP 0.5, 12</code>	
PLOT / DRAW	<code>PLOT x,y / DRAW dx,dy</code>	Plots a pixel at (x,y), or draws a line relative to the last plot.
	<code>PLOT 0,0 : DRAW 100,100</code>	
CIRCLE	<code>CIRCLE x, y, radius</code>	Draws a hollow circle.
	<code>CIRCLE 128, 96, 50</code>	
PEEK / POKE	<code>PEEK(addr) / POKE a,v</code>	Reads/writes bytes to the virtual 64K RAM space.
	<code>POKE 40000, 255</code>	

2. Beta BASIC 3 Structured Commands

These extensions bring modern, structured programming to standard BASIC, eliminating "spaghetti code."

Command	Syntax	Description & Example
---------	--------	-----------------------

PROC / ENDPROC	DEF PROC name(args)	Defines an isolated subroutine block.
	DEF PROC DRAW_MENU(X, Y) ... ENDPROC	
CALL	CALL name(args)	Executes a PROC (can also use PROC name()).
	CALL DRAW_MENU(10, 10)	
LOCAL / GLOBAL	LOCAL var1 / GLOBAL v	Restricts a variable's scope to the PROC, or pulls in a global.
	LOCAL TEMP_X	
WHILE / WEND	WHILE cond ... WEND	Pre-test conditional loop.
	WHILE HP > 0 : CALL COMBAT() : WEND	
REPEAT / UNTIL	REPEAT ... UNTIL cond	Post-test conditional loop (always runs at least once).
	REPEAT : GET A\$: UNTIL A\$ <> ""	
DO / LOOP	DO ... LOOP	Infinite loop block (use EXIT to break out).

		<code>DO : PRINT "IDLE" : LOOP</code>
EXIT	<code>EXIT [IF cond] [LOOP]</code>	Breaks out of a FOR , WHILE , REPEAT , or DO loop.
		<code>EXIT IF KEY\$ = "Q" DO</code>
CASE / WHEN	<code>CASE expr ... WHEN val</code>	Switch/case statement for multiple conditions.
		<code>CASE DIR : WHEN "N" : Y=Y+1 : ENDCASE</code>
GET	<code>GET var\$</code>	Pauses the program and waits for a single keystroke.
		<code>GET K\$</code>
LABEL	<code>LABEL "name"</code>	Creates a named string marker for GOTO or GOSUB .
		<code>LABEL "START"</code>

3. Extended Workstation Commands (Math, I/O & Multitasking)

Powerful commands adapted from Amstrad CPC, Locomotive, and Dartmouth Business BASIC.

Command	Syntax	Description & Example
---------	--------	-----------------------

AFTER / EVERY	<pre>EVERY sec, "label"</pre>	Spawns a background multitasking timer that triggers a GOSUB .
	<pre>EVERY 1.5, "UPDATE_CLOCK"</pre>	
MAT PRINT / READ	<pre>MAT PRINT array</pre>	Prints/Reads entire arrays at once without FOR loops.
	<pre>MAT PRINT GRID</pre>	
MAT ZER / CON	<pre>MAT array = ZER</pre>	Fills an entire array with 0s (ZER), 1s (CON), or Identity (IDN).
	<pre>MAT MAP = ZER</pre>	
MAT Math	<pre>MAT A = (scalar) * B</pre>	Multiplies an entire matrix by a constant. (Must use brackets!)
	<pre>MAT STATS = (2) * STATS</pre>	
OPEN # / CLOSE #	<pre>OPEN #id, "mode", "file"</pre>	Opens sequential files. Modes: "W" (Write), "R" (Read).
	<pre>OPEN #1, "W", "SAVE.DAT"</pre>	

PRINT # / INPUT # `PRINT #id; expr`

Writes/Reads data streams from files.

```
PRINT #1; SCORE
```

OBJECT `LET var =
OBJECT`

Instantiates a dynamic object. Properties are accessed with a dot.

```
LET P1 = OBJECT : P1.HP = 10
```

PLAY `PLAY "MML"`

Background music macro language (`Octave`, `Length`, `Tempo`).

```
PLAY "04 T120 C D E"
```

FORMAT USING `FORMAT USING
"fmt"; var`

Strictly formats a number and assigns it as a string to `var`.

```
FORMAT USING "###.00"; MONEY$
```

SET / CONS `SET var INT`

Enforces strong typing (`INT`, `FLOAT`) or creates a Constant.

```
CONS MAX_LIVES = 3
```

4. System Functions

Functions evaluate data and return a result. They are used within expressions (e.g. `LET X = SQR(9)`).

Math Functions

- **ABS(n)**: Absolute value.
- **SGN(n)**: Returns -1 (negative), 0, or 1 (positive).
- **INT(n) / FIX(n)**: Returns the integer floor, or truncates decimals.
- **SQR(n)**: Square root.
- **SIN(n), COS(n), TAN(n)**: Trigonometry (uses Radians).
- **MAX(a, b...), MIN(a, b...)**: Returns the largest/smallest value from a list.
- **ROUND(n, d)**: Rounds number *n* to *d* decimal places.
- **PI**: Constant 3.14159...

String & Input Functions

- **LEN(s\$)**: Returns the length of string *s\$*.
- **VAL(s\$) / STR\$(n)**: Converts a string to a number, or a number to a string.
- **CHR\$(n) / CODE(s\$)**: ASCII character from number, or ASCII number from character.
- **LEFT(s, n) / RIGHT(s, n) / MID(s, pos, len)**: Extracts portions of a string.
- **INKEY\$**: Instantly scans the keyboard (non-blocking). Returns " " if nothing pressed.
- **SPACE\$(n)**: Returns a string of *n* spaces.
- **TIME\$ / DATE\$**: Returns the system clock/date as a string.

6 Demonstration Programs

*Copy and paste these directly into the terminal, then type **RUN** to watch them execute.*

1. The Matrix Math Initializer (Dartmouth BASIC Style)

Demonstrates setting up a 3x3 grid, auto-filling it with an Identity matrix, scaling it via scalar multiplication, and printing it without any **FOR** loops.

```
Basic
10 CLS
20 PRINT "--- MATRIX MATH DEMO ---"
30 LET GRID(2, 2) = 0
40 MAT GRID = IDN
50 PRINT "1. IDENTITY MATRIX:"
60 MAT PRINT GRID
```

```
70 PRINT ""
80 PRINT "2. MULTIPLY BY 5:"
90 MAT GRID = (5) * GRID
100 MAT PRINT GRID
```

2. Time-Sliced Multitasking (Amstrad CPC Style)

Demonstrates the **EVERY** command running a background clock while the foreground runs an interactive drawing loop.

Basic

```
10 CLS : BORDER 1
20 LET TICKS = 0
30 EVERY 1.0, "CLOCK_TICK"
40 PRINT "BACKGROUND CLOCK STARTED."
50 PRINT "PRESS 'Q' TO QUIT."
60 DO
70 LET X = INT(RND() * 256)
80 LET Y = INT(RND() * 100)
90 INK (INT(RND() * 6) + 1)
100 CIRCLE X, Y, 10
110 PAUSE 2
120 EXIT IF INKEY$ = "Q" DO
130 LOOP
140 DI
150 INK 7 : PRINT AT 20,0; "PROGRAM HALTED."
160 END
170 LABEL "CLOCK_TICK"
180 LET TICKS = TICKS + 1
190 INK 7 : PRINT AT 0, 90; "TICK: "; TICKS
200 RETURN
```

3. Structured Data Objects

Showcases the **OBJECT** type for entity management instead of parallel arrays.

Basic

```
10 CLS
20 LET PLAYER = OBJECT
30 LET PLAYER.NAME = "HERO"
40 LET PLAYER.HP = 100
50 LET PLAYER.GOLD = 50
```

```

60 PRINT "PLAYER STATS:"
70 PRINT LINE PLAYER
80 PRINT "-----"
90 LET ENEMY = OBJECT
100 LET ENEMY.NAME = "GOBLIN"
110 LET ENEMY.HP = 30
120 PRINT "ENEMY ENCOUNTERED:"
130 PRINT LINE ENEMY

```

4. Advanced Geometric Graphics

Utilizes Beta BASIC graphics commands to draw nested shapes and arcs.

Basic

```

10 CLS : BORDER 0 : INK 7 : PAPER 0 : CLS
20 PRINT "DRAWING GEOMETRY..."
30 LET CX = 128 : LET CY = 96
40 FOR I = 10 TO 80 STEP 10
50 INK (I MOD 7) + 1
60 ELLIPSE CX, CY, I, I / 2
70 NEXT I
80 INK 2
90 BOX 48, 56, 208, 136
100 INK 7 : PRINT AT 22, 0; "DONE!"

```

5. Structured Procedures & Case Logic

Eliminates **GOTO** by using strict Beta BASIC 3 **PROC** routing and **CASE** evaluation.

Basic

```

10 CLS
20 PRINT "1. ROLL D6"
30 PRINT "2. ROLL D20"
40 PRINT "3. QUIT"
50 GET C$
60 CASE C$
70 WHEN "1"
80 PROC ROLL(6)
90 WHEN "2"
100 PROC ROLL(20)
110 WHEN "3"
120 END

```

```

130 DEFAULT
140 PRINT "INVALID KEY"
150 ENDCASE
160 PAUSE 50
170 GOTO 10
180 DEF PROC ROLL(SIDES)
190 LOCAL RESULT
200 LET RESULT = INT(RND() * SIDES) + 1
210 PRINT "YOU ROLLED A "; RESULT
220 ENDPROC

```

6. Sequential File I/O & Formatting

Demonstrates opening a disk stream, writing formatted strings to it, and reading them back.

Basic

```

10 CLS
20 PRINT "SAVING DATA TO DISK..."
30 OPEN #1, "W", "TEST_LEDGER"
40 LET BAL = 1234.56
50 FORMAT USING "£####.00"; BAL$
60 PRINT #1; "ACCOUNT"
70 PRINT #1; BAL$
80 CLOSE #1
90 PRINT "READING FROM DISK..."
100 OPEN #2, "R", "TEST_LEDGER"
110 INPUT #2; N, B
120 CLOSE #2
130 PRINT "NAME: "; N$
140 PRINT "BALANCE: "; B$

```